TECHNICAL PAPER | EDUCATIONAL & AUTHORIZED-TESTING USE ONLY | V1 · SEPTEMBER 2026

Trust Me, I'm a Camera

A man-in-the-middle and RTP replay attack against RTSP video-surveillance streams

Martín López Aranburu • linkedin.com/in/martín-lópez-aranburu • Barcelona Cybersecurity Congress, November 2026

ABSTRACT

RTSPoofing is a working man-in-the-middle tool that defeats the RTSP/RTP pipeline used by the large majority of consumer and small-business IP cameras and network video recorders (NVRs). By combining ARP-based network positioning with a byte-transparent RTSP relay and RTP-level frame substitution, an on-path attacker can silently replace a live camera feed with pre-recorded footage while the recording NVR continues to log a fully valid-looking stream — correct sequence numbers, correct timestamps, correct sender identity, and, optionally, a forged RTCP wall-clock. This paper documents the attack's three-stage architecture, its extension to UDP-transported RTP, a field observation about switch MAC-address-table learning that let the attack proceed on a network configured with Dynamic ARP Inspection, and the concrete defenses that close each stage.

KEYWORDS RTSP · RTP/RTCP · man-in-the-middle · ARP spoofing · video surveillance · IoT/embedded security · protocol authentication

Contents

1 Introduction 3

2 Background: RTSP and RTP 3
2.1 The RTP header . . . 3

3 Threat model and architecture 4

4 Phase I — network-layer positioning 5

5 Phase II — transparent interception 5
5.1 Byte-transparent RTSP relay, credentials included . . . 5
5.2 RTPProbe: live parameter detection . . . 5

6 Phase III — media substitution 6
6.1 Continuity rewrite . . . 6
6.2 Fixed-rate pacing . . . 6
6.3 Two independent notions of "now" . . . 6

7 Extending to UDP transport 7

8 Field observation: MAC-address-table learning 8

9 Empirical scope 9

10 Detection and countermeasures **10**

11 Discussion: assumptions and limitations **10**

12 Conclusion **10**

13 Ethics and responsible disclosure **11**

A Reference tables **11**
A.1 CLI reference 11
A.2 `.rtp` capture record layout 11

1 Introduction

Physical security infrastructure runs on network infrastructure, and the network infrastructure it runs on was never designed to authenticate itself.

The large majority of IP cameras and the NVRs that record them speak RTSP in plaintext, negotiate an RTP media session with no cryptographic integrity, and trust whatever arrives on the negotiated channel. This paper asks a narrow, concrete question: what does it actually take to make an NVR record a fabricated feed — without touching camera or NVR firmware, without credentials, and without the NVR ever registering an anomaly — and where, specifically, does that stop working?

The answer is implemented as **RTSPoofing**, an asynchronous Rust/Tokio proxy (~1,600 lines across eight modules) that performs the attack end-to-end against both transport modes RTSP supports. Sections 4–7 walk through its three-stage pipeline and its UDP extension in the order the implementation executes them. Section 8 documents an unplanned field finding — a switch's own MAC-learning behavior undermining a correctly configured Dynamic ARP Inspection policy on a segmented corporate network. Sections 9–11 scope what was and wasn't verified, and Section 10 maps each stage to a specific, low-cost defense.

> **SCOPE**
> All testing described here was performed in isolated lab environments or under explicit written authorization. Findings labeled *informal* or *field observation* come from the author's own test environments and are not a controlled study across a representative device population — see §9 and §11 for the specific caveats.

2 Background: RTSP and RTP

RTSP (Real Time Streaming Protocol, RFC 2326) is the out-of-band control channel almost every IP camera speaks: a client — here, the NVR — issues text requests to negotiate and control a media session.

```
NVR -> CAM  DESCRIBE rtsp://192.168.1.64/stream RTSP/1.0
CAM -> NVR  200 OK -- SDP body: available media, codecs, transport options

NVR -> CAM  SETUP rtsp://192.168.1.64/stream/track1 RTSP/1.0
              Transport: RTP/AVP;unicast;client_port=8000-8001
CAM -> NVR  200 OK -- Transport: ...;server_port=9000-9001  (or interleaved=0-1)

NVR -> CAM  PLAY rtsp://192.168.1.64/stream RTSP/1.0
CAM -> NVR  200 OK -- media now flowing
```

The `SETUP` exchange is the detail this paper returns to repeatedly: it negotiates one of two transports for the RTP/RTCP that follows. **TCP-interleaved** frames the media inside the same RTSP connection, each frame prefixed with a `$` marker, a channel number, and a 2-byte length (RFC 2326 §10.4). **UDP** instead sends RTP and RTCP as independent datagrams on the `client_port/server_port` pairs exchanged during `SETUP`. Both are common in the field; §7 covers why the UDP case needs separate handling.

2.1 The RTP header

Once negotiated, media flows as a sequence of RTP packets (RFC 3550). Three header fields carry the entire state a receiver uses to make sense of the stream:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|V=2|P|X|  CC   |M|     PT      |       Sequence Number         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                           Timestamp                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Synchronization Source (SSRC) identifier            |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Sequence Number	16-bit, increments per packet. RTP runs over lossy UDP by design, so conforming receivers conceal isolated gaps rather than terminating the session (RFC 3550 §8). A large or backward discontinuity is what actually surfaces as a defect.
Timestamp	32-bit media clock, conventionally 90 kHz for video regardless of frame rate. Should advance monotonically; a stall or reversal is what a jitter buffer notices.
SSRC	32-bit sender identity. A receiver is expected to treat a change as a new source and reset decoder state (RFC 3550 §8.2) — disruptive to conceal, not fatal to avoid.

None of these fields carry a signature or MAC. SRTP (RFC 3711) adds exactly that and is, practically, absent from consumer and SMB CCTV. Three unauthenticated fields are the entire surface the rest of this paper manipulates.

3 Threat model and architecture

The attacker is assumed to share a broadcast domain (an unsegmented VLAN) with the camera and the NVR — the common case among the deployments surveyed in §9 — and to hold no camera or NVR credentials. The goal is not to exfiltrate the stream but to substitute its *content* while the NVR continues to believe it is recording the camera live.

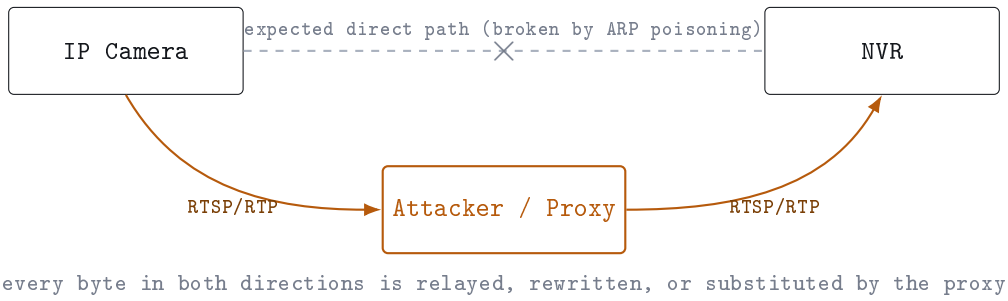


Figure 1: ARP spoofing does not open a new path — it breaks the one the camera and NVR believe they're using, and both sides' traffic is redirected through the attacker instead.

The implementation executes this as three stages, one `MITMSession` task per accepted TCP connection:

1. **Phase I — network-layer positioning** (§4): ARP spoofing places the attacker on-path at layer 2.
2. **Phase II — transparent interception** (§5): a byte-level RTSP/RTP relay with live parameter auto-detection.
3. **Phase III — media substitution** (§6): paced, continuity-preserving RTP frame injection.

Both supported transports (TCP-interleaved and UDP, §7) are routed through the same frame-handling function, `process_camera_frame()`, specifically so probing, capture, and injection behavior cannot silently diverge between them.

4 Phase I — network-layer positioning

Camera and NVR normally address each other directly. The attacker poisons both ARP caches with periodic forged replies — conventionally via a tool such as `arpspoof` — so each side resolves the other's IP to the attacker's MAC address:

```
arpspoof -i eth0 -t <NVR> <CAM>
arpspoof -i eth0 -t <CAM> <NVR>
```

With IP forwarding enabled on the attacker host, traffic continues to flow end-to-end and neither side's link appears to break — it now simply routes through the attacker first. This is a standard, well-documented technique; its only requirement is layer-2 adjacency, which is the normal case for camera and NVR on a flat, unsegmented network (§9).

> **WHAT STOPS IT**
> Dynamic ARP Inspection (DAI) validates ARP packets against a DHCP-snooping or static binding table and drops ones that don't match, which is a complete stop for this phase in isolation. §8 documents a case where DAI was correctly configured and this phase still succeeded — not because DAI failed, but because of an unrelated switch behavior.

5 Phase II — transparent interception

With traffic flowing through it, the proxy does two things concurrently: relay the RTSP control channel byte-for-byte, and passively characterize the RTP media it's relaying.

5.1 Byte-transparent RTSP relay, credentials included

An `RtspTextScanner` accumulates raw TCP bytes and emits complete RTSP messages as they become available — boundaries are the header block's terminating blank line plus any `Content-Length` body, exactly mirroring how a stateful `InterleavedParser` on the media side finds complete `$`-framed RTP frames. Every RTSP request and response is forwarded unmodified *except* where a UDP `SETUP` exchange requires a Transport-header rewrite (§7).

This includes any `Authorization` header. RTSP supports Basic and Digest authentication on these same requests, and a meaningful share of real deployments enable it — but the proxy never needs to know the camera's password. The NVR computes its own credentials exactly as it always does and sends them through a connection it believes is direct; the proxy simply relays that header like every other byte. Camera authentication protects against a new, unauthorized client opening a session. It does not protect a session between two already-authorized parties from an on-path relay that was never asked to authenticate at all.

5.2 RTPProbe: live parameter detection

Rather than requiring the operator to specify payload type, SSRC, or frame rate up front, the proxy derives them from the live stream during a short warm-up window. Each RTP frame updates a majority vote over payload type, SSRC, and (for the interleaved case) channel number; the value with the most occurrences by the time the window closes (a configurable

duration *and* a minimum frame count) is taken as ground truth — deliberately not just the first packet seen, since the packets immediately after `PLAY` are often noisy.

```
[00:00.0] Analyzing stream...
[00:01.2] PT=96 seen 847 times -> dynamic H264
[00:02.1] SSRC=0xDEAD1234 dominant (99.8% of frames)
[00:02.8] Estimated FPS: 25.0 Hz (via RTP timestamp deltas)
[00:03.0] Codec: H264 (fragmented FU-A detected)
[00:03.0] Probe complete -- starting injection
```

Frame rate specifically is derived without any out-of-band signaling, using only the fact that RTP's video clock runs at a fixed, standard 90,000 Hz regardless of the actual frame rate: sampling consecutive packets' RTP timestamp deltas and taking the *median* across a sliding window (up to the last 60 valid samples, filtering deltas that wrap, exceed one clock period, or arrive with an implausibly small wall-clock gap) converges on

```
fps = 90000 / dts

e.g.  3,600 ticks between two frames -> 90000 / 3600 = 25 fps
```

6 Phase III — media substitution

Once parameters are known, an injector replaces the live stream with frames read from a `.rtp` capture file — either recorded earlier from the same camera in the proxy's own `record` mode (an 8-byte record header plus raw RTP payload per frame, written on a dedicated blocking task so a slow disk never stalls the live relay, see Appendix A), or synthesized from any source video with `ffmpeg`.

6.1 Continuity rewrite

Every injected frame's sequence number and timestamp are rewritten to continue, monotonically, from the last values observed on the *live* stream at the moment injection starts, and the SSRC is set to the one the real camera was using (or an explicit operator override). The transition from live to injected is therefore invisible at the RTP-metadata level: there is no gap, no reversal, and no change of apparent source for a receiver to key off.

6.2 Fixed-rate pacing

Frames are emitted on a `tokio::time::interval` ticking once per `1/fps` seconds, configured with `MissedTickBehavior::Delay`: a slow write or scheduler hiccup delays the next tick rather than bursting subsequent frames to catch up. An unpaced injector that simply emits frames as fast as the network allows is, in practice, the easiest way for this kind of attack to get noticed — dropped or corrupted frames and visibly bursty inter-arrival timing are exactly the sort of signal that stands out even to something that never decodes a single frame.

6.3 Two independent notions of "now"

Cameras commonly carry two separate timestamps: an on-screen OSD burned into the pixels by camera firmware, and an RTCP Sender Report's NTP field — a wall-clock value some VMS or forensic export tooling reads independently of the picture. Spoofing the first is free: an injected clip carries whatever OSD timestamp it was recorded with. The proxy additionally fabricates its own SR every `rtcp_sr_interval` seconds, stamped with the real current time, tracking the injected stream's own packet and octet counts:

Bytes	Field	Content
0–4	`Header`	V=2, P=0, RC=0 · PT=200 (SR) · length=6 words
4–8	`SSRC`	Sender identifier — matches the injected stream's SSRC
8–12	`NTP seconds`	Wall-clock "now" at send time
12–16	`NTP fraction`	Sub-second wall-clock precision
16–20	`RTP timestamp`	Media-clock value at this instant, same clock as the stream
20–24	`Packet count`	Running total for this synthetic sender
24–28	`Octet count`	Running payload byte total

The more consequential finding here is not the RTCP mechanics but the ordering of what matters: spoofing the picture already defeats the audience that matters most — a human reviewing footage. RTCP timestamp forgery only matters against tooling that reads the wall clock instead of the frame; it is included for completeness against that narrower, more technical audience, not because it is the headline exposure.

7 Extending to UDP transport

Everything in §5–§6 was described for TCP-interleaved media. RTSP's `SETUP` just as commonly negotiates plain UDP — independent RTP/RTCP datagrams on their own ports — and that case needs different handling for a reason that's easy to gloss over: ARP spoofing places the attacker on-path at *layer 2*, but with standard IP forwarding enabled, a UDP datagram addressed to the NVR's or camera's real IP is routed transparently by the kernel at the IP layer without ever reaching the proxy's own sockets. Being on-path at L2 is not the same as being on-path at the application layer; injecting into UDP media requires actually owning the datagrams, which means acting at the layer where the transport itself is negotiated — the `SETUP` exchange.

The fix rewrites that negotiation the same way ARP spoofing rewrites the network below it, one layer up:

1. The NVR's `SETUP` declares `client_port=8000-8001`; the proxy substitutes ports it owns before forwarding the request to the camera.
2. The camera, none the wiser, now sends its RTP/RTCP straight to the proxy.
3. The camera's response declares `server_port=9000-9001`; the proxy substitutes ports it owns there too, and restores the NVR's own original `client_port` in the same response so the NVR isn't confused by a value it never sent.
4. The NVR, none the wiser, now sends its RTCP receiver reports straight to the proxy as well.

The rationale for four sockets rather than two: some NVR/camera implementations validate that received RTP/RTCP actually originates from the exact port pair declared during `SETUP`, so relaying or injecting media has to send from the socket each side was told about, not merely any socket that happens to reach them. This is a defensive design choice carried in the implementation — it was not verified against a specific catalog of NVR/camera firmware, a limitation restated explicitly in §11.

All four sockets feed the same `process_camera_frame()` path used for TCP-interleaved media, so probing, capture, injection, and RTCP forgery cannot silently diverge between the two transports — auto-detection, pacing, and forged timestamps behave identically regardless of which one a given camera happens to use.

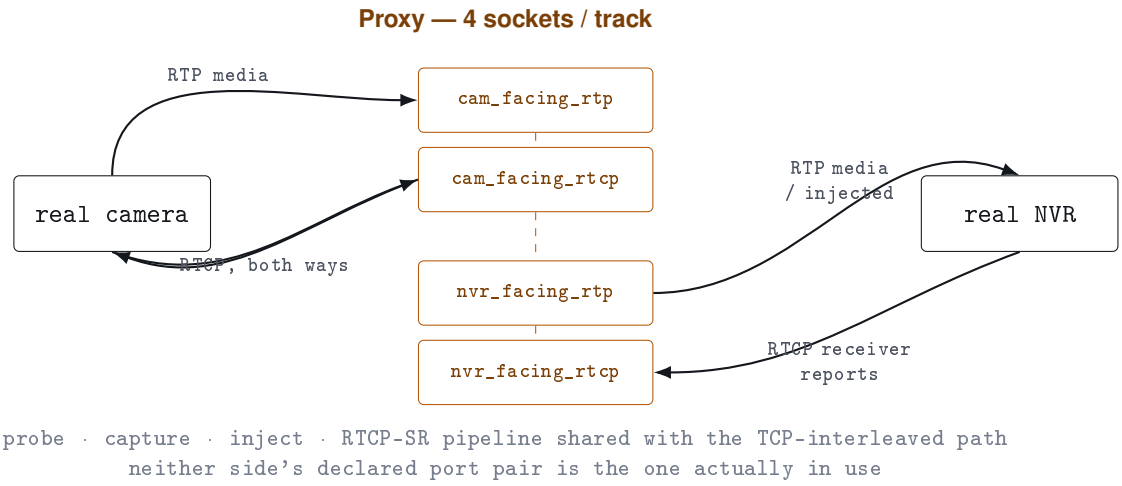


Figure 2: Four sockets, not two: each side's real port pair is rewritten to one the proxy owns, so both media *and* RTCP reports terminate at the proxy in both directions.

8 Field observation: MAC-address-table learning

One test network was a segmented, managed corporate environment with, as far as could be determined, Dynamic ARP Inspection enabled — the expectation going in was that forged ARP replies would simply be dropped and Phase I would go no further.

OBSERVED (HIGH CONFIDENCE)

After ARP spoofing began, a packet capture on the attacker host showed the switch's forwarding table relearn the camera's MAC address onto the attacker's switch port. The pre-existing camera↔NVR TCP session subsequently reset, and the NVR's own reconnect logic established a new session — this time through the proxy.

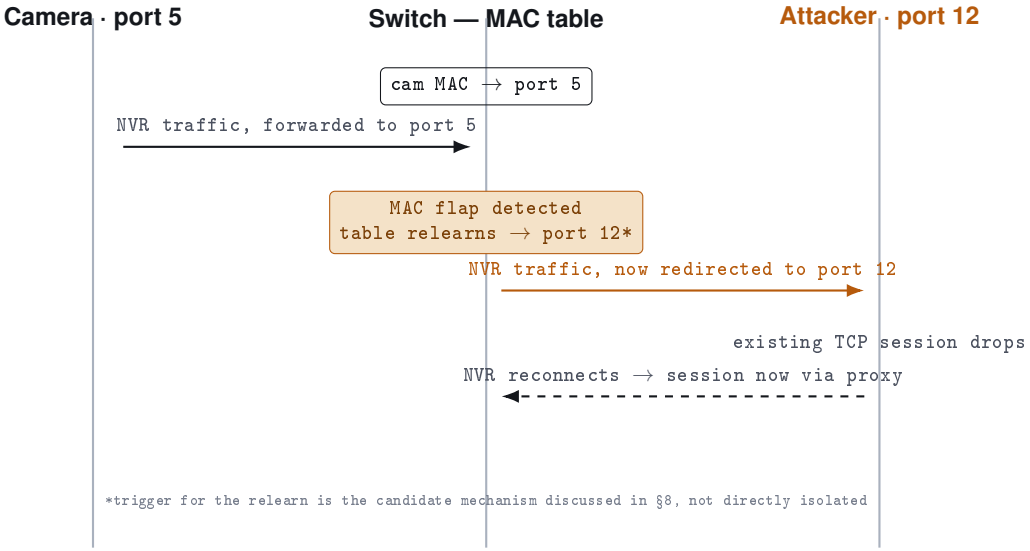


Figure 3: The observed event chain: a MAC-table relearn redirects NVR traffic to the attacker's port, the existing session drops, and the NVR's own reconnect logic re-establishes it through the proxy.

CANDIDATE MECHANISM (HEDGED)

MAC-address-table learning is a distinct, always-on layer-2 mechanism, unrelated to DAI's validation of ARP *payloads*: a switch continuously rewrites its forwarding entry for a MAC address to whichever port most recently produced a frame carrying that source address, regardless of whether the ARP traffic that provoked the situation was itself permitted. The most plausible trigger here is frames carrying the camera's real source MAC arriving via the attacker's port during the ARP-cache transition window, causing the table entry to flap and resettle on the attacker's port. This was not isolated with a second capture on the switch itself, so this section reports the *effect* with confidence and the mechanism as the best-supported explanation available — not a confirmed root cause.

What's actionable regardless of the exact trigger: a correctly configured security control was circumvented by an entirely separate, unrelated feature of the same device. DAI did precisely what it was designed to do — validate ARP payloads. Something else on the same switch, operating at a different layer of the same forwarding pipeline, produced the same outcome anyway. §10 turns this into a specific audit item rather than a general "enable DAI" recommendation.

9 Empirical scope

Across the author's own test environments:

Environment	Result	Condition
Home networks	**Reliably works**	Consumer router, no DAI
Small business	**Usually works**	Unmanaged switch, no hardening
Dedicated CCTV/NVR	**Often works**	Flat network, consumer-grade gear
Segmented corporate	**Not observed to work**	DAI enforced & MAC-learning that doesn't undermine it

CAVEAT, STATED PLAINLY

These are informal observations across the environments the author tested, not measurements from a controlled study across a representative device population. Treat category boundaries as directional, not statistical.

10 Detection and countermeasures

Stage	Defense	Why it works
Phase I	**Dedicated VLAN** for camera traffic	Removes the flat-network adjacency §4 and §9 depend on.
Phase I	**DAI, audited** — not just enabled	§8: confirm MAC-learning behavior doesn't quietly reroute traffic to an unrecognized port during a flap; toggling DAI on is necessary but, per §8, not sufficient on its own.
Phase II–III	**RTSP over TLS** (RTSPS)	Removes the proxy's ability to read or rewrite the plaintext control channel that §5–§7 all depend on — the single defense that breaks the whole chain at once.
All stages	**ARP / network anomaly monitoring**	Detective control (arpwatch, NDR/XDR) for cases the preventive controls above miss or are misconfigured.

11 Discussion: assumptions and limitations

- **Transport mix.** UDP is common in the field but not universal — some deployments default to TCP-interleaved specifically to avoid UDP loss surfacing as visible glitches. TCP-only coverage is not sufficient in general; the reverse claim ("UDP dominates") reflects the author's direct observation, not a survey.

- **Receiver loss tolerance favors the attacker.** Because RTP receivers conceal isolated gaps by design (§2), the continuity requirements in §6.1 are about avoiding a *visible* discontinuity, not a hard technical requirement for the session to survive at all — the attack surface is more forgiving than a literal reading of "must stay continuous" would suggest.

- **Four-socket rationale is a design assumption, not a confirmed finding.** §7 states the source-port-validation rationale as the reason for the design; which specific NVR/camera implementations actually enforce it remains an open question for anyone extending this work.

- **RTCP-SR consumption by VMS/forensic tooling** (§6.3) is plausible and consistent with the field's documented purpose (RFC 3550 §6.4.1), but was not confirmed against a specific named product.

- **§8's mechanism is a candidate explanation**, not a verified root cause — restated here deliberately, since it's the paper's most novel claim and the one most worth an independent replication with switch-side capture.

12 Conclusion

- IP cameras and NVRs trust anything that speaks TCP to them on the right port.

- RTSP/RTP carries no built-in stream authentication; SRTP exists and is essentially absent from this market segment.

- The attack works without touching camera or NVR firmware, and without camera credentials.

- Detection from the NVR's own vantage point alone is close to impossible without indepen-

dent network-level monitoring.

- Most of the home and SMB installs surveyed were vulnerable as shipped, with default configuration.

If a security system can be defeated by a few thousand lines of an evening's work, the sticker on the box was never the whole story.

13 Ethics and responsible disclosure

All testing described in this paper was conducted in isolated lab environments or under explicit, written authorization from the network owner. The implementation is released for research and authorized security testing only. Testing a network without the owner's explicit authorization is illegal in most jurisdictions and is not endorsed by this work in any form; the author bears no responsibility for uses inconsistent with this statement.

A Reference tables

A.1 CLI reference

Flag	Default	Purpose
`--mode`	—	`record` or `inject`
`--camera-host`	—	Camera IP/hostname the proxy forwards to
`--listen-port`	554	Port the proxy listens on for the NVR
`--capture-file`	captured.rtp	Binary `.rtp` record/replay file
`--inject-delay`	3.0s	Warm-up before injection begins
`--inject-fps`	25.0	Target injection frame rate
`--inject-ssrc`	auto	Override detected SSRC (hex or decimal)
`--inject-only`	false	Suppress the real stream entirely
`--rtcp-sr-interval`	5.0s	Synthetic RTCP SR cadence (0 disables)
`--tcpkill-iface`	—	Interface for forcing an already-established session to reconnect

A.2 `.rtp` capture record layout

Bytes	Field
0–4	Record length, big-endian u32 (= 2 + 2 + payload length)
4	Interleaved channel
5	Reserved (0)
6–8	Payload length, big-endian u16
8…	Raw RTP payload